AutoEncoder-Compressed Parallel Split Learning for Pre-trained Model Fine-Tuning

Bas Meuwissen · **Vasileios Tsouvalas** · Nirvana Meratnia

Eindhoven University of Technology

4th Workshop on Advancements in Federated Learning (WAFL), ECML-PKDD 2026

01 Context & approach

The challenge

Parallel split learning offloads model computation but exchanges activations and gradients at every step. Compressing pretrained features with an unaligned autoencoder can destroy task accuracy. *AE-PSL* aligns the compressor before fine-tuning to reduce communication while preserving useful representations.

Our contribution

Compress activations and gradients through a lightweight split-layer autoencoder.
Align to public pretrained features, then adapt to each client's distribution.
Evaluate global and local inference across four vision benchmarks.

1 Align to pretrained features

Train the autoencoder on public ImageNet100 features with the pretrained model frozen.

2 Adapt to each client

Warm up locally, retain client-specific encoders, and average decoders into one shared server decoder.

3 Fine-tune with compression

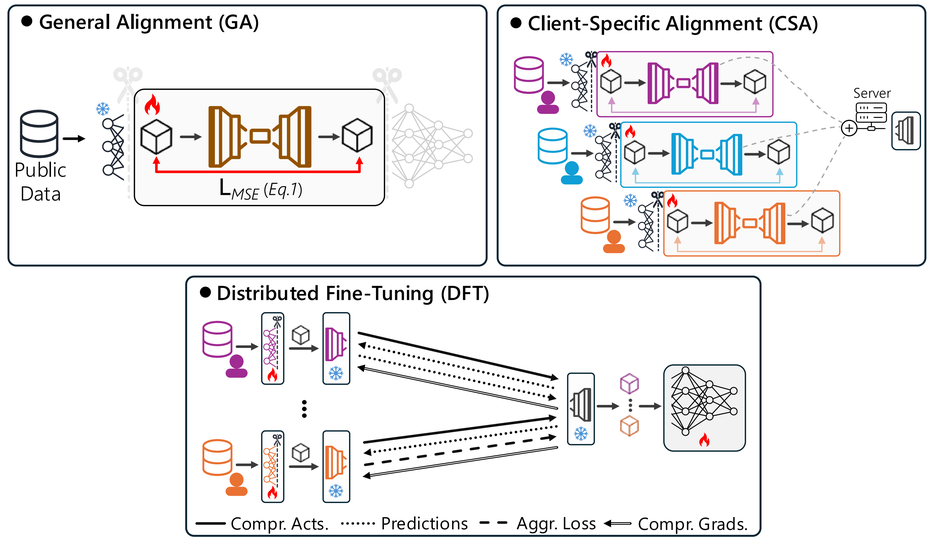
Freeze the autoencoder; exchange compressed features and gradients. Classification tokens bypass compression; labels remain on clients.

Scope

Validated for supervised ViT-B/32 vision fine-tuning; multimodal and language adaptation remain untested. Requires public-data alignment and local warm-up; global and local inference use different compression settings.

02 Method

Align learned compression with pretrained features before distributed fine-tuning.



Core formulation

$$\mathcal{L}_{\text{rec}} = \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \|H(x) - D_\psi(E_\phi(H(x)))\|_2^2$$

H: frozen pretrained features; E/D: encoder/decoder. Minimize reconstruction error on public data, then each client's data.

Paper

Code

03 Results

≈10×
communication reduction
Relative to uncompressed split fine-tuning.

82.7 ± 0.2%
CIFAR-100 accuracy
5 clients; matches the uncompressed reference.

Accuracy at approximately 10× communication reduction

Method	CIFAR-100	Food101	SUN397	FEMNIST
No compression (R=1)	82.7 ± 0.2	69.1 ± 0.9	67.0 ± 0.6	87.9 ± 0.3
C3-SL	50.9 ± 1.2	40.3 ± 1.2	41.3 ± 2.1	83.5 ± 0.1
Rand-Top-K	76.2 ± 0.4	60.2 ± 0.2	62.9 ± 1.0	87.2 ± 0.4
ADC	68.4 ± 0.4	54.6 ± 1.6	52.9 ± 1.1	81.1 ± 0.9
AE-PSL (Ours)	**82.7 ± 0.2**	**69.6 ± 1.2**	**66.7 ± 0.3**	**88.0 ± 0.1**

Top-1 accuracy (%), mean ± SD, 3 seeds. ViT-B/32; 5 clients; compressed methods R≈10. First three datasets: global inference, AE bypassed. FEMNIST: local inference, AE active.

Alignment prevents feature mismatch

Ablation	CIFAR-100	FEMNIST
Unaligned AE	1.4 ± 0.3	86.9 ± 2.0
+ General alignment	75.2 ± 0.4	87.1 ± 1.1
+ Freeze AE	79.6 ± 0.9	88.0 ± 0.6
+ Client alignment (Ours)	**82.2 ± 0.7**	**88.0 ± 0.7**

Accuracy (%), reported ± values, 3 seeds; ViT-B/32, 5 clients, R=15. CIFAR-100: global; FEMNIST: local inference. Cumulative ablations; final row is *AE-PSL*.

Takeaway

AE-PSL preserves uncompressed CIFAR-100 accuracy at **≈10× lower communication**, while outperforming the tested compression baselines. Alignment and freezing are crucial; global inference bypasses the autoencoder, whereas local FEMNIST inference retains it.

On efficacy of Meta-Learning for Domain Generalization in Speech Emotion Recognition

Raeshak King Gandhi · **Vasileios Tsouvalas** · Nirvana Meratnia

Eindhoven University of Technology

PerFail · IEEE PerCom Workshops 2023 · pp. 421-426

01 Context & approach

The challenge

Speech emotion models often fail on unfamiliar speakers, languages and corpora. *DA-MLDG* tests whether weighting learning by estimated shifts between training domains improves recognition on a completely held-out corpus.

Our contribution

Estimate domain shift from the gap between average energy scores.
Use that gap to weight the meta-test loss during training.
Compare seven transfer scenarios and expose remaining generalization failures.

1 Split the source corpora

Sample disjoint meta-train and meta-test domains; keep the final target corpus outside training.

2 Adapt and measure shift

Take a meta-train gradient step, then measure the absolute energy-score gap between the two source subsets.

3 Optimize for transfer

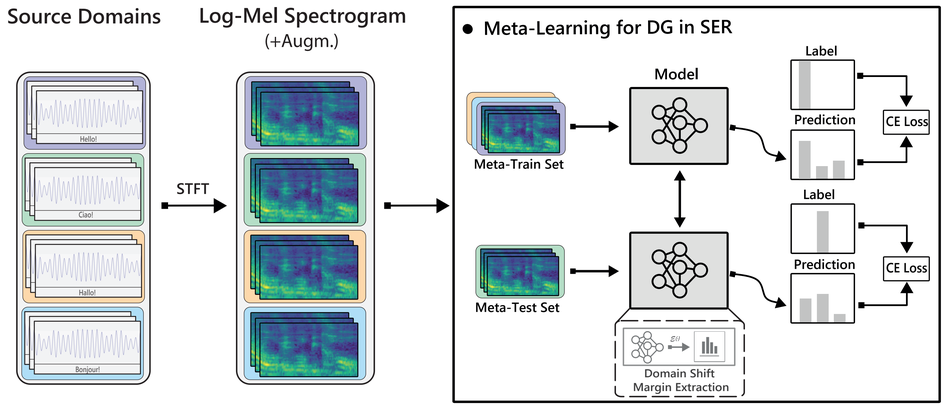
Combine meta-train loss with the gap-weighted meta-test loss; update the model and resample the source split.

Scope

Transfer gains are uneven; the reported performance remains insufficient for practical use. Small corpora, missing classes and no trial dispersion limit comparisons.

02 Method

Domain-aware meta-learning exposes persistent limits in cross-corpus speech emotion recognition.



Core formulation

$$\mathcal{L} = \mathcal{L}_{\theta}(\mathcal{S}') + \gamma \mathcal{L}_{\theta'}(\mathcal{V})$$

S′ and V are disjoint source subsets. θ′ follows a meta-train update; γ controls the weight of its meta-test loss.

$$\gamma = \left| \mathcal{E}(\mathcal{S}', p_{\theta'}) - \mathcal{E}(\mathcal{V}, p_{\theta'}) \right|$$

E is the absolute mean log-sum-exp of temperature-scaled logits. A larger source-domain gap gives the meta-test loss more weight.

Paper

Code

03 Results

4 / 7
above supervised Deep-All
Log-mel transfer scenarios

2 / 7
highest among four methods
Log-mel scenarios 4 and 5

Cross-domain emotion recognition

Scenario / target	Deep-All	RSC	MLDG	DA-MLDG (Ours)
1 / IEMOCAP	26.95	32.36	27.61	**31.35**
2 / RAVDESS	43.40	44.78	38.86	**39.41**
3 / TESS	47.03	43.77	46.29	**46.80**
5 / AESDD	60.65	56.69	60.45	**62.26**
6 / ShEMO	31.54	40.70	37.87	**38.79**

Held-out target accuracy (%), mean of ten trials; log-mel inputs with a VoxCeleb-pretrained ResNet-18.

Transfer with pretrained audio embeddings

Scenario / target	Deep-All (%)	DA-MLDG (%) (Ours)
1 / IEMOCAP	28.57	**32.11**
2 / RAVDESS	42.61	**39.82**
5 / AESDD	61.17	**64.11**
6 / ShEMO	33.16	**40.22**

The same source/target scenarios using TRILLsson embeddings; mean target accuracy (%) over ten trials.

Takeaway

Transfer improves unevenly: *DA-MLDG* beats supervised Deep-All in four of seven log-mel scenarios and leads all four methods in two. With pretrained embeddings, it beats the supervised baseline in **only three of seven** scenarios.

DeltaMask: Minimizing Communication in Federated Fine-Tuning of Vision Foundation Models

Vasileios Tsouvalas · Yuki M. Asano · Aaqib Saeed

Eindhoven University of Technology, The Netherlands | University of Technology Nuremberg, Germany

IEEE BigData 2024

01 Context & approach

The challenge

Federated adaptation of large vision models is expensive even with one-bit updates. *DeltaMask* keeps pretrained weights fixed and communicates selected changes in learned binary masks, reducing the information sent each round.

Our contribution

Adapt frozen vision models through stochastic binary masks.
Select informative mask changes and compress their indices with probabilistic filters.
Measure accuracy and bitrate across eight datasets and multiple pretrained architectures.

1 Learn masks

After one linear-probing round, freeze the backbone and optimize mask probabilities on each client.

2 Select and compress changes

Compare local and shared-seed server masks. Rank changed positions by KL divergence, then encode selected indices with Binary Fuse filters and DEFLATE.

3 Reconstruct and aggregate

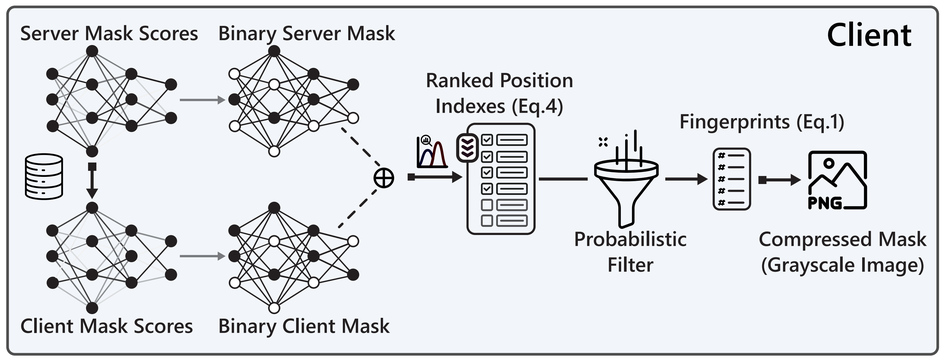
The server queries filter membership, flips the indicated mask bits, and updates global probabilities with Bayesian aggregation.

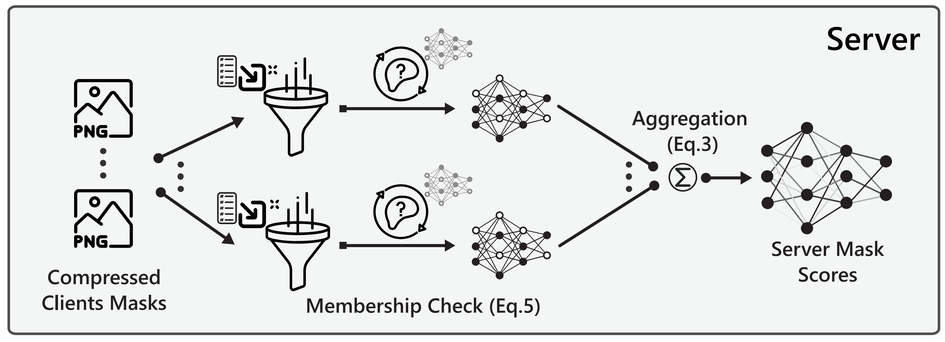
Scope

Probabilistic filters can add false-positive mask changes; accuracy remains below unrestricted fine-tuning. Evaluation covers image classification; broader task transfer is untested.

02 Method

Transmit compressed changes in stochastic masks instead of full pretrained weight updates.





Core formulation

$$\widetilde{w} = m \odot w_{\text{pre}}, \qquad m_i \sim \text{Bernoulli}(\theta_i)$$

w_{pre}: frozen pretrained weights; θ: learned keep probabilities; m: sampled binary mask. Adapt which weights are active.

$$\Delta = \{i : m_i^{\text{local}} \neq m_i^{\text{server}}\}, \qquad \Delta' = \text{TopK}_{\text{KL}}(\Delta)$$

Δ: changed mask indices; K: retained count. Rank changes by local-to-server Bernoulli KL divergence, then send selected indices.

Paper Code

03 Results

0.151 bpp
Average transmitted bitrate
8 datasets · CLIP ViT-B/32 · IID

85.44%
Average accuracy
Same setting · full participation

Low bitrate with a measured accuracy trade-off

Method	Avg. accuracy (%)	Avg. bits/parameter
Fine-tuning	88.08	32
FedMask	79.23	1.0
EDEN	84.14	0.691
FedPM	86.07	0.872
DeltaMask (Ours)	**85.44**	**0.151**

CLIP ViT-B/32 · 8 datasets · IID Dir(10) · 30 clients · full participation · 100 rounds. Means of 3 runs.

Non-IID data with 20% participation

Method	Avg. accuracy (%)	Avg. bits/parameter
Fine-tuning	83.19	32
EDEN	78.56	0.715
FedPM	81.64	0.948
DeltaMask (Ours)	**81.13**	**0.233**

Same 8 datasets and backbone · non-IID Dir(0.1) · 30 clients · 20% participation · 300 rounds. Means of 3 runs.

Takeaway

DeltaMask sends **selected mask changes**, reaching **0.151 bits/parameter** at **85.44% accuracy** across eight IID tasks. Compared with FedPM at 0.872 bits/parameter, it reduces communication while trailing accuracy by 0.63 percentage points.

EFU: Enforcing Federated Unlearning via Functional Encryption

Samaneh Mohammadi · **Vasileios Tsouvalas** · Iraklis Symeonidis · Ali Balador · Tanir Ozcelebi · Francesco Flammini · Nirvana Meratnia

RISE Research Institutes of Sweden | Mälardalen University | Eindhoven University of Technology

ACM CIKM 2025

01 Context & approach

The challenge

A client's unlearning update matters only if it enters aggregation. *EFU* binds encrypted learning and unlearning updates to an agreed aggregate, while a local objective balances forgetting with retained utility and limited parameter drift.

Our contribution

Bind encrypted updates to an authorized aggregation function.
Combine forget loss, adversarial-example training, and inverted MAS regularization.
Compare sample-, class-, and task-level forgetting with full retraining.

1 Learn or unlearn locally

Train normally or unlearn with the three-term loss; task forgetting also optimizes retained-task loss. Generate adversarial examples once offline.

2 Cluster and encrypt

Compress weights into centroids. Send encrypted centroids, the visible position mapping, and a partial functional key for the round.

3 Decrypt the agreed aggregate

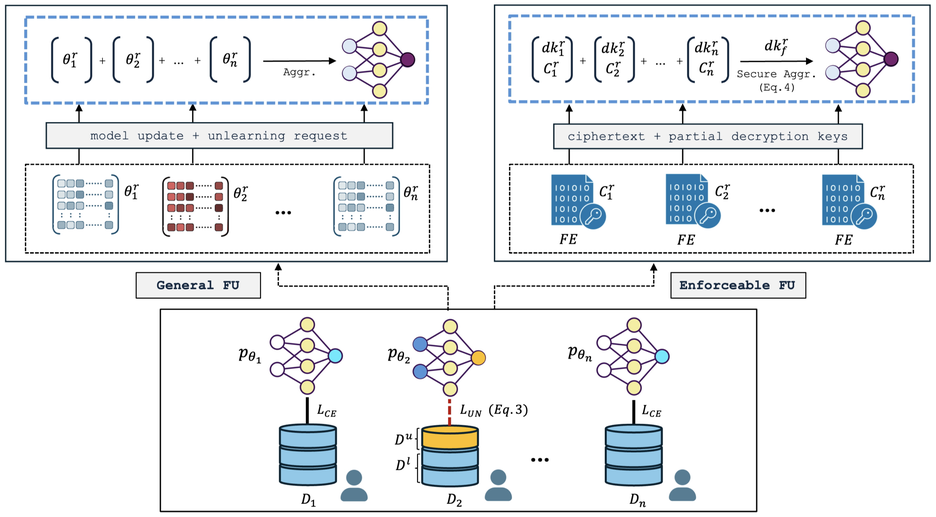
Expand encrypted centroids, combine key shares, and recover the authorized weighted aggregate for the next round.

Scope

Security depends on trusted clients, valid key shares, and the stated indistinguishability assumptions. Low forget accuracy is behavioral evidence. Authorized aggregation does not prove complete erasure or compel model deployment.

02 Method

Bind learning and unlearning updates to an authorized encrypted aggregate.



Core formulation

$$\mathcal{L}_{\text{UN}} = -\mathcal{L}_{\text{CE}}(D^u) + \mathcal{L}_{\text{CE}}(D^{u,\text{adv}}) + \sum_j \tilde{\Omega}_j(\theta_j - \tilde{\theta}_j)^2$$

D^u: forget data; $D^{u,\text{adv}}$: adversarial variants; $\tilde{\theta}$: starting weights; $\tilde{\Omega}$: inverted MAS importance. Forget while limiting unrelated parameter drift.

$$\theta_j^{r+1} = \text{Dec}(\{\theta_{i,\text{enc}}^r[j]\}_i, \text{dk}_f)$$

Encrypted client updates and the function key dk_f yield the agreed aggregate. The binding concerns decryption, not subsequent deployment.

Paper **Code**

03 Results

95.37%
Retained gender accuracy
LFW · task forgetting

5.12%
Forgotten identity accuracy
Same *EFU* model

Class forgetting with retained utility

Method	Retain accuracy (%)	Forget accuracy (%)
Full retrain	94.82	13.72
PGD	91.82	14.31
FedOSD	93.06	12.77
EFU + FedOSD (Ours)	**92.73**	**12.54**
EFU (Ours)	**92.43**	**13.06**

CIFAR-10 · ResNet-18 · forget 10% of classes · 10 clients (1 unlearning) · 20% participation · Dir(0.1). 100 rounds; 10 unlearning rounds from round 50. Means of 3 runs; higher retain / lower forget is better.

Forget identity, retain gender

Method	Gender accuracy (%)	Identity accuracy (%)
Full retrain	96.86	4.71
PGD	94.81	10.32
FedOSD	95.02	5.01
EFU (Ours)	**95.37**	**5.12**

LFW · ViT-B/32 · 30 clients, all unlearning identity · full participation. Ten unlearning rounds from round 50; means of 3 runs.

Takeaway

EFU retains **95.37% gender accuracy** while identity accuracy falls to **5.12%** in LFW task forgetting. Its **authorized encrypted aggregation** incorporates the agreed updates, while complete erasure and subsequent model deployment remain separate requirements.

EncCluster: Scalable Functional Encryption in Federated Learning through Weight Clustering and Probabilistic Filters

Vasileios Tsouvalas · Samaneh Mohammadi · Ali Balador · Tanir Ozcelebi · Francesco Flammini · Nirvana Meratnia

Eindhoven University of Technology, The Netherlands | RISE Research Institutes of Sweden, Sweden | Mälardalen University, Sweden

Pervasive and Mobile Computing, 2025

01 Context & approach

The challenge

Encrypting every model weight makes secure federated aggregation expensive. *EncCluster* encrypts a small set of representative centroids and compresses the position mapping, reducing client encryption work and transmitted data.

Our contribution

Combine weight clustering with decentralized multi-client functional encryption.
Encode weight-to-centroid assignments using Binary Fuse filters.
Evaluate accuracy, upload volume, and encryption time across four architectures.

1 Train and cluster

Train locally, then group similar weights into a compact set of centroids and record their assignments.

2 Encrypt the compact update

Encrypt centroid values. Encode position-cluster pairs with a Binary Fuse filter and send them with partial functional keys.

3 Reconstruct and aggregate

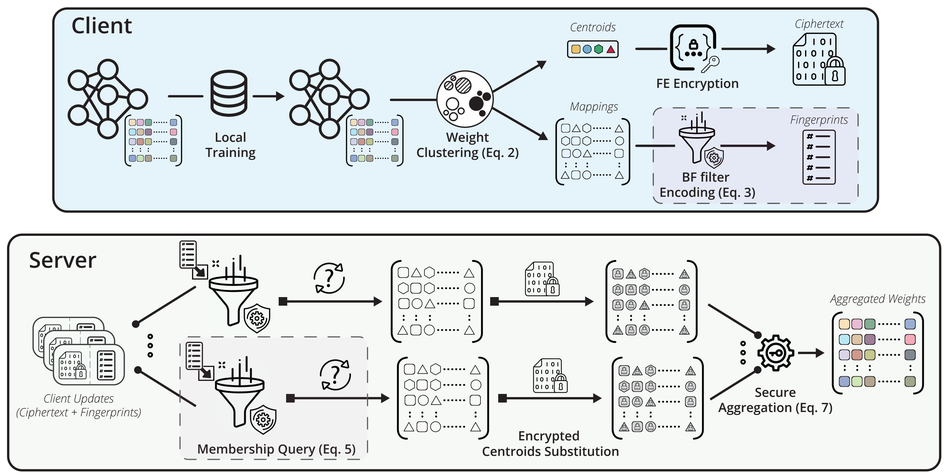
Recover the mapping, expand encrypted centroids into model positions, and decrypt the authorized weighted aggregate.

Scope

Assumes an honest-but-curious setup authority/server and secure channels; poisoning and denial-of-service are outside scope. Filter errors do not provide differential privacy. Simulation timings are hardware-specific; repeated-run uncertainty is unreported.

02 Method

Encrypt compact centroids, then reconstruct encrypted weights for secure aggregation.



Core formulation

$$\mathcal{L}_{\text{cluster}} = \sum_{i=1}^{d} (\theta_i - z_{a_i})^2$$

θ_i: weight; z: centroids; a_i: assigned cluster. Approximate many weights with a few shared values before encryption.

$$\theta_j^{r+1} = \frac{1}{|D|} \text{Dec}(\{\widehat{\theta}_{n,j}^{r}\}_n, \text{dk}_f)$$

Encrypted weights are preweighted by client sample count. Divide the decrypted sum by total samples |D|; dk_f authorizes aggregation.

Paper

Code

03 Results

0.284×
Upload volume / FedAvg
CIFAR-10 · full participation

2.04 s
Client encryption time
A10 / 96-core cluster simulation

Lower encrypted communication and computation

Method	IID Δ (pp)	Non-IID Δ (pp)	Upload / FedAvg	Encrypt (s)
FedAvg + clustering	-0.12	-0.71	0.034	—
DeTrust-FL	-0.09	-0.10	3.743	329.53
SEFL	-0.98	-1.74	2.581	7.64
EncCluster (Ours)	**-0.32**	**-0.79**	**0.284**	**2.04**

CIFAR-10 · ResNet-20 · 30 clients · full participation · 100 rounds. Dir(10)/Dir(0.1), 128 centroids, 256-bit keys. Deltas vs. FedAvg; timings from an A10 / 96-core cluster simulation.

Architecture coverage

Architecture	C10 Δ (pp)	C100 Δ (pp)	Encrypt (s)
ConvNeXt-Tiny	-0.36	-1.29	2.02
ResNet-20	-0.32	-1.19	2.03
MobileNet	-0.24	-0.97	2.08
ConvMixer-256/8	-0.19	-1.02	2.12

IID CIFAR-10/100 · 10 clients · full participation · 100 rounds. Accuracy changes use each architecture's FedAvg baseline; same cluster hardware.

Takeaway

EncCluster reduces uploads to **0.284× FedAvg** with a **0.32-point IID accuracy decrease** on CIFAR-10. Client encryption takes **2.04 s** in the A10 / 96-core cluster simulation, versus 7.64 s for SEFL.

FedCode: Communication-Efficient Federated Learning via Transferring Codebooks

Saeed Khalilian · **Vasileios Tsouvalas** · Tanir Ozcelebi · Nirvana Meratnia

Eindhoven University of Technology, The Netherlands

IEEE EDGE 2024

01 Context & approach

The challenge

Clustering model weights reduces value storage, but transmitting every weight index still costs bandwidth. *FedCode* usually exchanges only cluster centers in both directions. Periodic calibration also transfers compressed assignments to realign client and server models.

Our contribution

Reduce upstream and downstream traffic by exchanging compact codebooks.
Map existing weights to received centers without sending every index each round.
Use periodic calibration to control the communication-accuracy trade-off.

1 Broadcast centers

Cluster global weights and send sorted centers. Clients map existing weights to their nearest received centers.

2 Train and return codebooks

Clients train and cluster the full model. The server concatenates received codebooks and maps its weights to the nearest centers.

3 Calibrate periodically

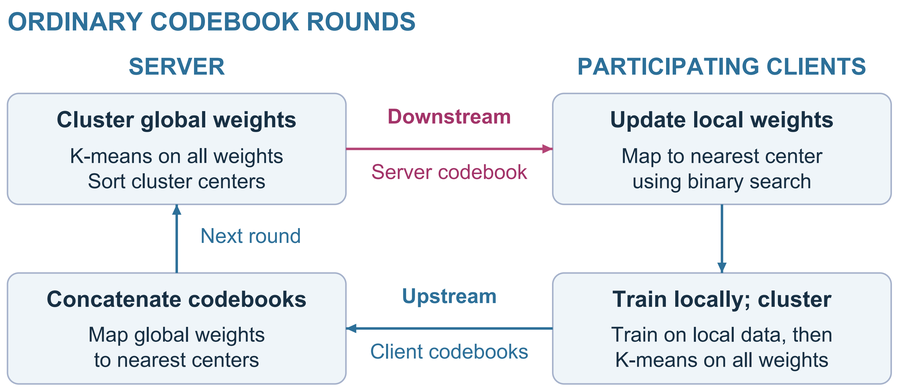
Also transfer compressed weight indices on calibration rounds. Reconstruct models; server-side calibration then applies FedAvg.

Scope

Communication volume is measured; client latency, energy and run-to-run uncertainty are not established. Calibration direction labels conflict in the paper; F1/F2 are retained without assigning directions.

02 Method

Exchange codebooks in both directions and periodically calibrate weight assignments.



Codebook = cluster centers; ordinary rounds omit weight indices.

PERIODIC CALIBRATION — SELECTED ROUNDS

Transfer codebook + weight indices in the relevant direction.

Clients reconstruct weights from indices and centers.

The server reconstructs client models, then applies FedAvg.

Core formulation

$$j^* = \arg\min_j |w_i - c_j|, \qquad w_i \leftarrow c_{j^*}$$

w_i: existing weight; c_j: received center. Ordinary rounds choose the nearest center; the server uses concatenated client codebooks.

$$\mathrm{DTR} = \frac{2Pb}{2Kb + (F_1 + F_2)P\lceil \log_2 K \rceil}$$

P weights, K centers, b bits/weight; F1 and F2 are calibration frequencies. Less index transfer reduces traffic.

Paper Code

03 Results

14.6×
joint communication reduction
IID CIFAR-100; ResNet-20.

58.07%
FedCode test accuracy
FedAvg: 61.22% on the same workload.

Compress both communication directions

Method	Accuracy (%)	Down DTR	Up DTR	Joint DTR
FedAvg	61.22	—	—	—
OMC-T	61.21	—	2.9	1.4
FedCompress	59.67	4.11	4.11	4.11
FedZip	57.98	—	19.8	1.9
FedCode (Ours)	**58.07**	**24.2**	**10.4**	**14.6**

IID CIFAR-100, ResNet-20; 10 clients, full participation, 100 rounds, 4 local epochs. *FedCode*: 64 centers, F1=0.2, F2=0.5. DTR = FedAvg bits / method bits; higher is better.

Calibration controls the trade-off

F1 / F2	Accuracy Δ (pp)	Joint DTR	Data (MB)
1 / 1	-0.86	5.3	394.21
0.2 / 0.5	-3.15	14.6	143.92
0.1 / 0.1	-21.64	44.3	47.35
0 / 0	-46.88	264.7	7.9

Same CIFAR-100/ResNet-20 setting; 64 centers. Accuracy change is relative to FedAvg; negative means lower accuracy. FedAvg communication: 2102 MB.

Takeaway

FedCode achieves **14.6× less joint communication** at 58.07% accuracy, 3.15 percentage points below FedAvg in this setting. Without calibration, communication falls further but accuracy drops 46.88 points below FedAvg.

Communication-Efficient Federated Learning through Adaptive Weight Clustering and Server-Side Distillation

Vasileios Tsouvalas · Aaqib Saeed · Tanir Ozcelebi · Nirvana Meratnia

Eindhoven University of Technology

IEEE ICASSP 2024

01 Context & approach

The challenge

Compressing client models saves upload bandwidth, but averaging their weights breaks the shared-centroid structure. *FedCompress* restores compression at the server and adjusts the number of centroids as learning progresses, reducing traffic in both directions.

Our contribution

Compress client uploads and server broadcasts with weight clustering.
Adapt cluster capacity using the effective rank of local embeddings.
Measure accuracy, communication and model size across five vision and audio tasks.

1 Cluster client weights

After supervised warm-up, combine label training with centroid clustering and upload the compressed model.

2 Restore server compression

Apply FedAvg, then distill its aggregated model into a clustered student using auxiliary unlabeled data.

3 Adapt capacity

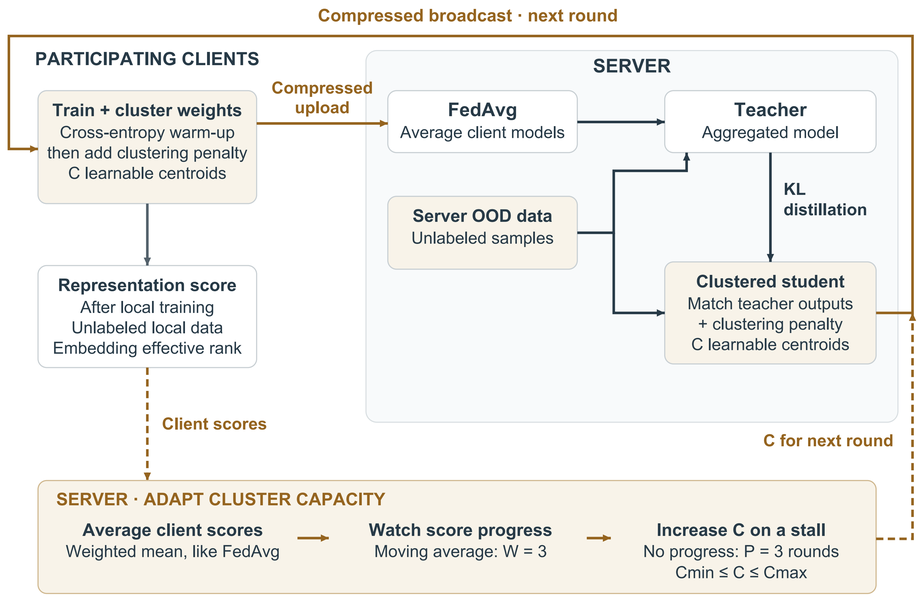
Combine clients' effective-rank scores; increase the cluster count for later rounds when score progress stalls.

Scope

Server distillation requires auxiliary data and extra optimization. Simulated clients; two-run accuracy means without reported dispersion.

02 Method

Compress both communication directions through local clustering and server distillation.



Core formulation

$$\mathcal{L}_S = \mathcal{L}_{\mathrm{KL}}(p_T \| p_\theta) + \beta \mathcal{L}_{\mathrm{wc}}(\theta, \mu, C)$$

Teacher p_T guides student p_θ; the clustering penalty groups weights around C centroids μ. β balances compression and imitation.

$$\mathcal{E}_k = \exp\left(-\sum_j r_j \log r_j\right)$$

r_j are normalized singular values of client k's embeddings. Their effective rank tracks representation diversity and guides cluster capacity.

Paper

Code

03 Results

5.41×
communication reduction
VoxForge versus FedAvg

-0.31 pp
accuracy difference
Same VoxForge comparison

Communication savings with small accuracy losses

Dataset	FedAvg (%)	Δ acc. (pp)	Comm. (×)	Model (×)
CIFAR-10	86.26	-1.83	4.53	5.18
CIFAR-100	60.68	-1.88	3.80	3.93
PathMNIST	88.22	-1.72	4.79	5.27
SpeechCommands	95.75	-0.42	5.04	5.09
VoxForge	81.05	-0.31	5.41	5.64

20 clients, 20 rounds, 10 local/server epochs; two-run mean accuracy. Signed accuracy changes and reduction ratios are relative to FedAvg.

Why compress again at the server?

CIFAR-100 method	Δ acc. (pp)	Comm. (×)	Model (×)
FedZip	-2.57	1.94	2.11
Without server compression	-2.67	1.02	1.62
FedCompress (Ours)	**-1.88**	**3.80**	**3.93**

CIFAR-100 under the same protocol; accuracy changes and reduction ratios relative to FedAvg.

Takeaway

FedCompress reduces communication across all five tasks, with **small accuracy losses**. On CIFAR-100, **server compression** raises the communication reduction from 1.02× to 3.80×.

Labeling Chaos to Learning Harmony: Federated Learning with Noisy Labels

Vasileios Tsouvalas · Aaqib Saeed · Tanir Ozcelebi · Nirvana Meratnia

Eindhoven University of Technology, The Netherlands

ACM TIST · Vol. 15, No. 2, Article 22 · 2024

01 Context & approach

The challenge

Devices can hold labels of very different quality. *FedLN* offers three alternative ways to handle this noise: correct labels before training, add teacher guidance during local learning, or reduce noisy clients' influence during aggregation.

Our contribution

Estimate client label noise from embeddings or prediction confidence.
Offer three alternatives: *NNC*, *AKD* and *NA-FedAvg*.
Test diverse synthetic and human-annotated noise in vision and audio.

A NNC: correct labels

Use a pretrained encoder and local nearest-neighbor voting to replace mismatched labels before federated training.

B AKD: guide local learning

Use estimated noise to activate extra teacher supervision from pretrained embeddings or global-model outputs.

C NA-FedAvg: reweight clients

Estimate noise once after warm-up, then give noisier clients less weight during server aggregation.

Scope

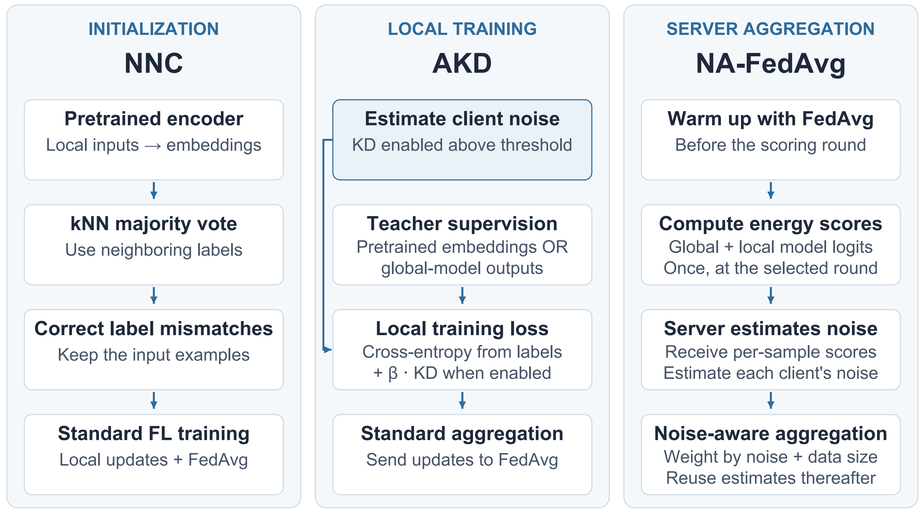
The best intervention depends on the noise pattern; *NNC* can fail under class flipping. Pretrained encoders add cost; three-run tables omit dispersion and do not establish equal computation.

02 Method

Three alternative interventions for client-dependent label noise in federated learning.

Three alternatives for handling noisy labels

Each acts at a different stage of federated learning.



KD = knowledge distillation

Core formulation

$$\mathcal{L}_{\text{AKD}} = \mathcal{L}_{\text{CE}} + \beta u_{\varepsilon}(n_l^m)\mathcal{L}_{\text{KD}}$$

For client m, estimated noise n_l^m gates teacher guidance through u_ε. β controls its strength alongside label-based cross-entropy.

$$\gamma_m = (1 - n_l^m)\frac{N_m}{N}$$

NA-FedAvg multiplies client m's data share N_m/N by its estimated clean-label fraction $1\text{-}n_l^m$.

Paper Code

03 Results

74.76%
NNC under random noise
CIFAR-10; 70% noise

54.59%
NNC under class flipping
CIFAR-10; 70% noise

Accuracy depends on the noise pattern

Method	Random noise (%)	Class flipping (%)
FedAvg	57.65	63.54
FedCorr	67.27	65.23
NNC (Ours)	**74.76**	**54.59**
AKD · embeddings (Ours)	**68.43**	**68.39**
NA-FedAvg (Ours)	**65.34**	**67.92**

CIFAR-10/ResNet-20: 70% label noise on 80% of 30 clients; random versus class-flipping noise. Three-run mean accuracy after 200 rounds.

Dependence on clients with clean labels

Noisy clients	NNC (%) (Ours)	NA-FedAvg (%) (Ours)
25%	**73.80**	**75.99**
50%	**74.54**	**73.27**
75%	**73.50**	**70.99**
100%	**73.76**	**64.02**

CIFAR-10: 40% random label noise within each noisy client; 30 clients, 200 rounds, three-run mean accuracy.

Takeaway

No single intervention wins everywhere: *NNC* leads the selected random-noise comparison but falls below FedAvg under class flipping. *NA-FedAvg* becomes less effective as **clean clients disappear**.

Privacy-preserving Speech Emotion Recognition through Semi-Supervised Federated Learning

Vasileios Tsouvalas · Tanir Ozcelebi · Nirvana Meratnia

Eindhoven University of Technology

EmotionAware · IEEE PerCom Workshops 2022

01 Context & approach

The challenge

Speech emotion models need costly annotations, and central training requires collecting sensitive recordings. Federated self-training uses each device's unlabeled speech alongside its small labeled subset, while exchanging model updates.

Our contribution

Combine spectral, temporal and channel attention in a speech CNN.
Filter local pseudo-labels using a device-specific confidence schedule.
Compare four speaker/client splits at the same 10% label budget.

1 Represent emotional speech

Convert local audio to log-mel features; an attention CNN focuses on spectral, temporal and channel cues.

2 Learn from unlabeled clips

Train jointly on true labels and confident pseudo-labels; filtering accounts for training progress and each client's participation.

3 Aggregate model updates

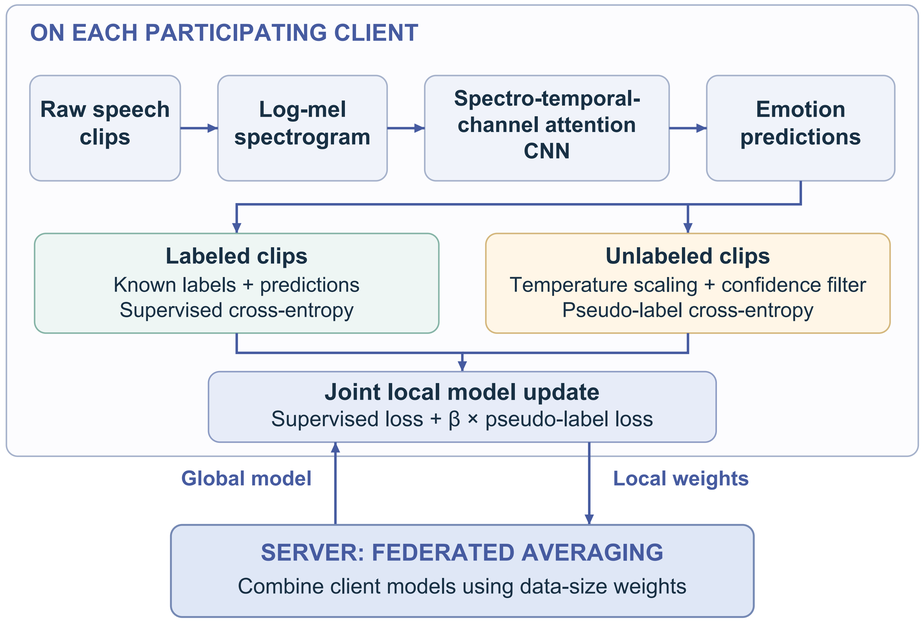
FedAvg combines local parameters. Raw speech stays on clients; no server training dataset is required.

Scope

One four-emotion corpus; random/per-speaker splits also differ in client count, and results omit dispersion. Raw-data locality does not provide differential privacy or protection against model-update attacks.

02 Method

Federated self-training improves speech emotion recognition when only 10% of recordings are labeled.



Core formulation

$$\mathcal{L}_k = \mathcal{L}_s(\mathcal{D}_L^k) + \beta \mathcal{L}_u(\mathcal{D}_U^k)$$

Client k combines supervised loss on labeled speech D_L^k with loss on confident pseudo-labels from D_U^k; β sets their relative influence.

Paper

Code

03 Results

10%
labeled speech
Remaining clips used unlabeled

56.83%
unweighted accuracy
Unseen speakers; per-speaker clients

Unlabeled speech helps at a fixed label budget

Speaker/client split	Supervised 10%	Self-training 10% (Ours)	Supervised 100%
Unseen / random	46.72	**53.94**	60.52
Unseen / per speaker	46.55	**56.83**	59.55
Overlap / random	51.58	**59.04**	64.78
Overlap / per speaker	49.97	**59.72**	62.79

IEMOCAP improvised speech, four emotions; mean unweighted accuracy (%) over five runs per split. Random/per-speaker partitions use 10/8 clients.

Fully labeled reference models

Same architecture	Unseen speakers (%)	Overlapping speakers (%)
Centralized	62.58	69.04
Federated	60.52	64.78

100% labels, random partitions; mean unweighted accuracy. Federated setup: 10 clients, 100 rounds, 80% participation, one local step.

Takeaway

Self-training improves every 10%-label comparison, while remaining below the corresponding fully labeled reference. It keeps **raw recordings local**; the experiments establish no formal privacy guarantee.

Federated Self-training for Semi-supervised Audio Recognition

Vasileios Tsouvalas · Aaqib Saeed · Tanir Ozcelebi

Eindhoven University of Technology, The Netherlands

ACM TECS · Vol. 21, No. 6, Article 74 · 2022

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

01 Context & approach

The challenge

Most audio collected on devices has no labels. *FedSTAR* lets this unused audio contribute to a shared model by combining labeled examples with confident local predictions used as temporary labels.

Our contribution

Learn jointly from labeled and unlabeled audio on each client.
Filter pseudo-labels with temperature scaling and an increasing confidence threshold.
Test label budgets, client distributions and optional public-data pretraining.

1 Share the current model

Broadcast one global model to clients; optional public-audio pretraining supplies its initial weights.

2 Combine two learning signals

The same local model predicts pseudo-labels, filters uncertain examples, and combines supervised and weighted pseudo-label losses in one backward pass.

3 Average and repeat

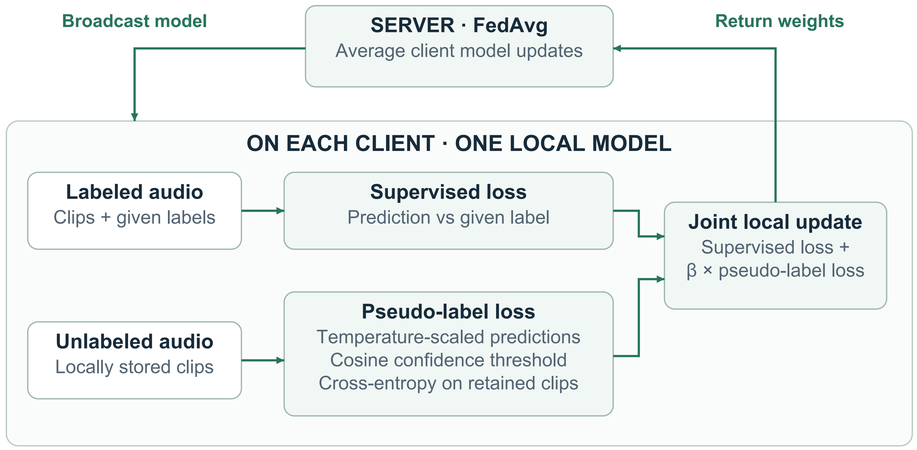
Use FedAvg to combine model updates and begin the next round; raw downstream audio stays local.

Scope

Gains vary by task and label budget; public-data pretraining does not always improve final accuracy. Client training is simulated; class-skew tests retain randomly distributed unlabeled clips.

02 Method

Use confident pseudo-labels to learn from audio that remains unlabeled on devices.



Compute both losses before the same backward update.

Repeat across selected clients and federated rounds. Raw downstream audio stays local.

Core formulation

$$\mathcal{L}_k = \mathcal{L}_{\mathrm{CE}}(y, p_{\theta^k}(x_l)) + \beta \mathcal{L}_{\mathrm{CE}}(\hat{y}, p_{\theta^k}(x_u))$$

One client model uses true labels y and confident pseudo-labels ŷ. β weights the unlabeled-audio loss; both terms share one backward pass.

Paper

Code

03 Results

3%
labeled data
15-client comparison

86.82%
Speech Commands accuracy
FedSTAR at 3% labels

Learning with 3% labels

Dataset	Supervised FL (%)	FedSTAR (%) (Ours)
Ambient Context	33.03 ± 0.002	**49.54 ± 0.005**
Speech Commands	62.98 ± 0.003	**86.82 ± 0.006**
VoxForge	54.26 ± 0.002	**55.82 ± 0.011**

3% labels, 15 clients, 100 rounds, 80% participation. Three-run mean accuracy; printed ± terms are described as variance.

Pretraining across label budgets

Labels	Random init. (%)	Pretrained (%)
3%	85.40 ± 0.008	85.17 ± 0.001
5%	88.52 ± 0.005	88.28 ± 0.002
20%	92.24 ± 0.012	93.67 ± 0.005
50%	92.26 ± 0.005	93.83 ± 0.003

Speech Commands: 50% of remaining unlabeled clips, 15 clients, 100 rounds. Three-run means; ± terms denote reported variance.

Takeaway

At 3% labels, **self-training improves all three tasks**, but the gains vary widely. Public-data pretraining can help early convergence, while its **final-accuracy benefit depends on the data budget**.

On the Effectiveness of Invisible Backdoor Attacks in Federated Learning

Mattjis Marinus · **Vasileios Tsouvalas** · Nirvana Meratnia

Eindhoven University of Technology, The Netherlands

UbiSec 2024 · Proceedings published 2025

01 Context & approach

The challenge

Invisible triggers must survive aggregation with honest updates. This study measures when existing backdoor attacks remain effective in federated learning and whether low-activation placement improves success without making the attack easier to detect.

Our contribution

Compare four invisible attack families with visible BadNet triggers.
Measure attack success, clean accuracy, visual distance, and detection scores.
Propose and evaluate low-activation trigger placement for LSB attacks.

1 Run a controlled comparison

Honest clients train normally; attackers use clean and poisoned samples. Aggregate updates across repeated federated rounds.

2 Vary attack conditions

Compare LSB, WaNet, Encoder, and LIRA with BadNet. Vary attacker prevalence and evaluate selected server defenses.

3 Test low-activation placement

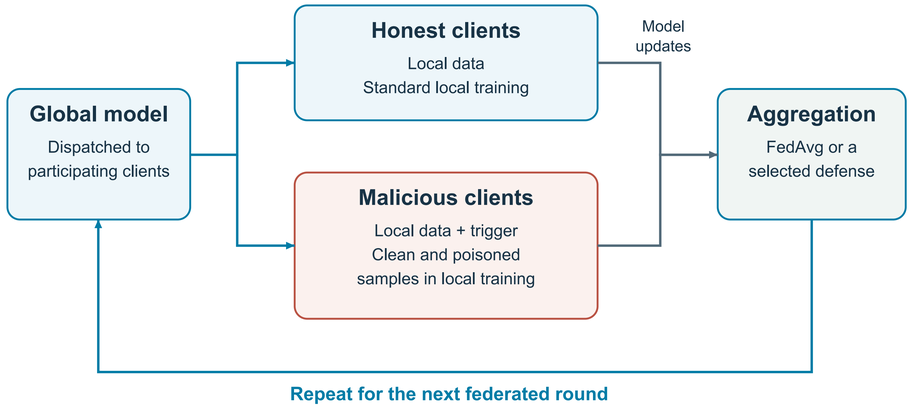
Place LSB triggers in low-activation regions, then compare attack success and detection scores with unmasked and high-activation controls.

Scope

Experiments use ResNet-18 and small image datasets; run count and the meaning of ± are unspecified. Placement evidence covers LSB. Greater visual similarity does not imply evasion of model-level detection.

02 Method

Evaluate subtle triggers, attacker prevalence, and the effectiveness-detectability trade-off.



Core formulation

$$\mathcal{L}_{\text{attack}} = \mathcal{L}_{\text{clean}} + \mathcal{L}_{\text{poison}\to t}$$

The two cross-entropy terms fit clean labels and poisoned samples labelled as target class t. Honest updates compete with this objective.

$$\text{ASR}(\%) = 100\,\frac{N_{\text{triggered}\to t}}{N_{\text{triggered}}}$$

$N_{\text{triggered}}$: poisoned test samples excluding the target class; the numerator counts those predicted as target t. ASR measures attack success.

Paper

Code

03 Results

88.59%
Encoder attack success
20% attackers · CIFAR-10

+19.80 pp
LSB700 ASR difference
Low minus unmasked placement

Attacker prevalence changes attack success

Attack	Clean acc., 10%	ASR, 10%	ASR, 20%
BadNet	84.00 ± 0.31	94.96 ± 0.81	97.52 ± 0.40
LSB	83.37 ± 0.33	41.59 ± 6.19	65.43 ± 3.93
WaNet	83.33 ± 0.31	44.41 ± 3.50	82.05 ± 3.59
Encoder	83.44 ± 0.27	29.44 ± 5.90	88.59 ± 4.01
LIRA	83.71 ± 0.41	10.52 ± 1.22	41.57 ± 4.66

CIFAR-10 · ResNet-18 · FedAvg · 10 clients · 100 rounds · Dir(0.5). Poison rate 0.5; target class 5. Columns give attacker fraction; reported ± is undefined.

Low-activation placement has a cost

LSB700 placement	ASR (%)	LPIPS	Anomaly index
Unmasked	41.59 ± 6.19	7.83e-5	1.61
Low (Ours)	**61.39 ± 4.47**	**2.25e-4**	**3.04**
High	36.89 ± 4.26	3.75e-4	2.01

LSB700 · CIFAR-10 · 10% attackers. Lower LPIPS means closer images; Neural Cleanse flags anomaly above 2. “Low” is the proposed placement.

Takeaway

Encoder reaches **88.59% ASR with 20% attackers**, showing the importance of attacker prevalence. Proposed **low-activation LSB700 placement** gains 19.80 points over unmasked placement, but raises Neural Cleanse’s anomaly index from 1.61 to 3.04.

Many-Task Federated Fine-Tuning via Unified Task Vectors

Vasileios Tsouvalas · Tanir Ozcelebi · Nirvana Meratnia

Eindhoven University of Technology

FedGenAI @ IJCAI 2025

01 Context & approach

The challenge

When clients hold multiple tasks, local updates can interfere within and across clients. Maintaining a separate model for every task increases storage and communication. *MaTU* transfers one unified task vector per client with lightweight adjustments that preserve task identity.

Our contribution

Represent multiple tasks with one vector, binary task masks and scalar rescalers.
Combine same-task updates and transfer information between tasks with aligned signs.
Avoid retaining client-specific model weights at the server across rounds.

1 Unify local tasks

Fine-tune LoRA parameters per task. Elect a shared sign and the largest aligned magnitude for each vector entry.

2 Preserve task identity

Attach a binary mask and scalar rescaler per task; transmit them with the unified vector.

3 Share related updates

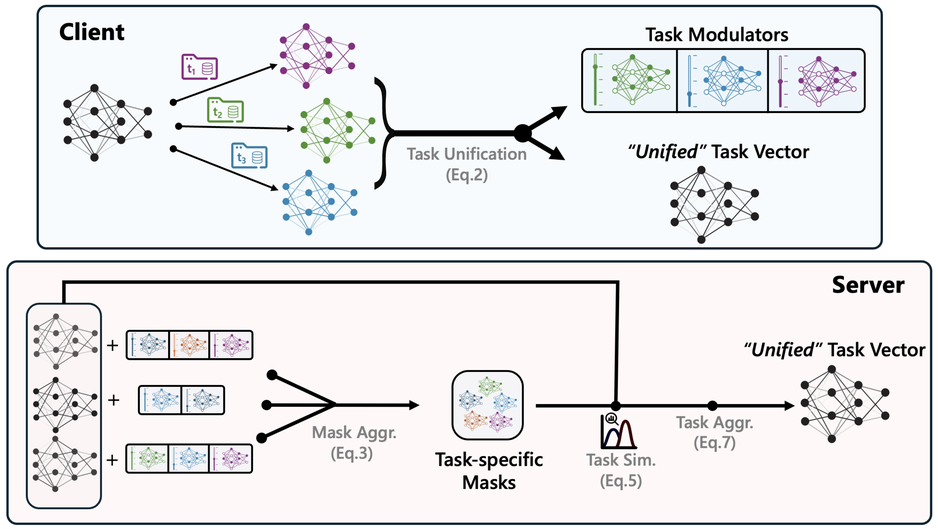
Aggregate same-task updates, combine sign-aligned tasks, and return unified vectors with updated modulators for the next round.

Scope

Vision classification with ViT-B/32 and LoRA; other modalities and backbones remain untested. No uncertainty reported; the eight-task MaT-FL baseline uses 300 rounds versus 100 for *MaTU*.

02 Method

Unify client task vectors while retaining task-specific directions and magnitudes.



Core formulation

$$\tau = \sigma \odot \mu$$

τ: unified task vector; σ: sign of the summed task vectors; μ: largest magnitude agreeing with that sign, entry by entry.

$$\dot{\tau}_t = \lambda_t m_t \odot \tau, \qquad \theta_t = \theta_p + \dot{\tau}_t$$

Mask m_t keeps task-aligned entries; scalar λ_t rescales them. Add the recovered task vector to pretrained weights θ_p.

Paper

Code

03 Results

79.47%
average task accuracy
8 tasks; multiple tasks per client.

8.04 Mbit/task
communication per round
MaT-FL: 20.43 Mbit/task/round.

Multiple tasks per client: accuracy and communication

Method	Avg. accuracy (%)	Mbit/task/round
Individual centralized	90.21	—
FedAvg	51.63	20.43
FedPer	48.37	17.16
MaT-FL	67.60	20.43
MaTU (Ours)	**79.47**	**8.04**

8 vision tasks; ViT-B/32, LoRA rank 16. 30 clients, 1 local epoch, task concentration ζt=0.5, 20% participation. 100 rounds; MaT-FL: 300. No uncertainty reported.

Scaling to the 30-task benchmark

Method	Normalized accuracy (%)
MaT-FL	52.62
MaTU (Ours)	**77.40**

30 tasks; accuracy normalized to individual task fine-tuning. ViT-B/32, 30 clients, 300 rounds, 1 local epoch, task concentration ζt=0.2, full participation.

Takeaway

MaTU reaches **79.47% accuracy at 8.04 Mbit/task/round** across eight tasks, exceeding MaT-FL with 100 versus 300 training rounds. Across 30 tasks, normalized accuracy rises from 52.62% to 77.40%, while remaining below individual task fine-tuning.

Fine-tuning Multimodal Transformers on Edge: A Parallel Split Learning Approach

Timo Fudala · **Vasileios Tsouvalas** · Nirvana Meratnia

Eindhoven University of Technology

FedGenAI @ IJCAI 2025

01 Context & approach

The challenge

Multimodal transformers are costly to fine-tune on edge devices. Even parameter-efficient federated methods execute the backbone locally. *MPSL* keeps lightweight image, audio and text tokenizers on clients while a shared server encoder performs the expensive computation.

Our contribution

Extend parallel split learning to paired modalities with one shared encoder.
Aggregate client losses for one server backward pass while raw labels stay local.
Evaluate task performance and client cost across seven multimodal datasets.

1 Tokenize and fuse

Clients tokenize each modality. Early fusion joins client tokens; late fusion joins representations after server encoding.

2 Train through one server

Return predictions for local loss computation. Aggregate losses, backpropagate once, then return cut-layer gradients for tokenizer updates.

3 Construct the evaluation model

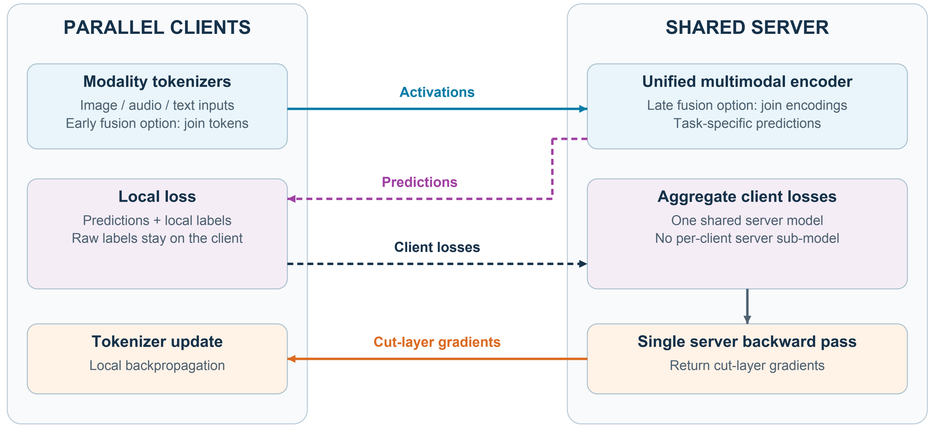
After training, average client tokenizers and join them with the shared server encoder.

Scope

Client savings shift computation to the server; classification performance varies by task. Retrieval trails centralized training; missing-modality robustness is untested.

02 Method

Keep multimodal tokenization on clients and move transformer fine-tuning to a shared server.



Core formulation

$$\mathcal{L}_S = \sum_{n \in \mathcal{N}} \frac{|\mathcal{B}_n|}{|\mathcal{B}|} \mathcal{L}_{C_n}$$

L_{Cn}: local client loss; B_n: client batch; B: combined batch. Minimize this batch-weighted loss in one server backward pass.

Paper

Code

03 Results

0.1 GFLOPs
average client compute
Per input, five classification tasks; FedAvg: 25.8.

58.2 ± 0.9%
MELD accuracy
100 clients; FedAvg: 48.9 ± 0.0%.

Client efficiency with task-dependent accuracy

Method	COCO-QA	UCF101	MELD	Client GFLOPs
Centralized	61.7 ± 0.1	91.3 ± 0.2	61.4 ± 0.4	25.8
FedAvg	53.5 ± 0.2	89.8 ± 0.9	48.9 ± 0.0	25.8
FedCLIP	13.1 ± 2.6	56.4 ± 1.2	50.7 ± 0.7	25.8
MPSL (Ours)	**46.2 ± 0.5**	**89.3 ± 1.0**	**58.2 ± 0.9**	**0.1**

Accuracy (%), reported ± values, 3 seeds. Distributed methods: 100 clients, Dir(0.1); Meta-Transformer ViT-B/16, last 6 blocks trained. Client GFLOPs/input averaged across five classification tasks.

Image-to-text retrieval remains challenging

Method	MS-COCO R@1	Flickr30K R@1	Client GFLOPs
Centralized	56.8 ± 2.4	66.8 ± 5.3	8.65
FedAvg	1.5 ± 0.1	11.2 ± 1.7	8.65
FedCLIP	0.1 ± 0.0	0.1 ± 0.0	8.65
MPSL (Ours)	**21.1 ± 0.8**	**26.0 ± 0.5**	**0.1**

Image-to-text recall@1 (%), reported ± values, 3 seeds. Distributed methods: 100 clients; full ViT-B/16 encoder fine-tuning. GFLOPs are per client input.

Takeaway

MPSL reduces average client compute to **0.1 GFLOPs per input**, but classification gains depend on the task: MELD improves over FedAvg while COCO-QA declines. Retrieval exceeds the tested federated baselines yet remains well below centralized training.

Plug-and-Play Multilingual Few-shot Spoken Words Recognition

Aaqib Saeed · **Vasileios Tsouvalas**

Eindhoven University of Technology, The Netherlands

arXiv:2305.03058 · 2023

01 Context & approach

The challenge

Adding words or languages to a fixed-vocabulary recognizer usually requires retraining. *PLiX* learns an acoustic embedding space where a few example recordings define new words at inference time, while the encoder stays fixed.

Our contribution

Train spoken-word embeddings on over 12 million clips in 20 languages.
Test unseen words, five unseen languages and language-identification episodes.
Measure separate small/base encoders on three edge devices.

1 Learn from episodes

Train an encoder on support/query tasks; training and test vocabularies remain disjoint.

2 Average support embeddings

For each new word, average its labeled support embeddings to form a prototype; keep the encoder frozen.

3 Compare each query

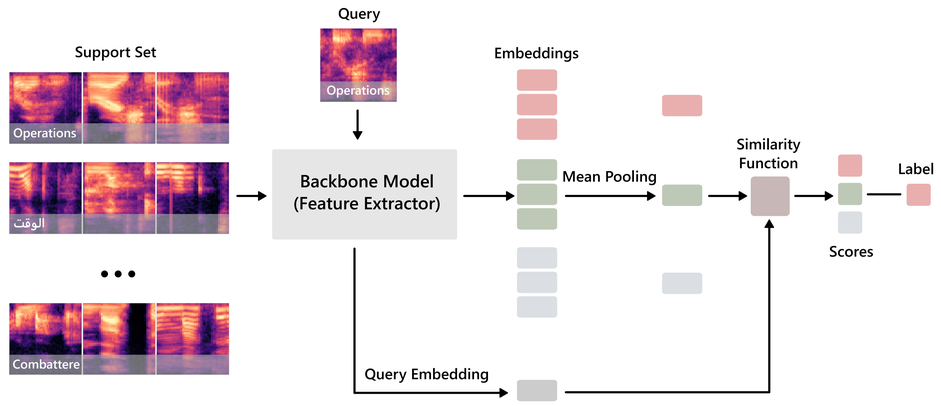
Encode the query directly and choose its nearest prototype by squared Euclidean distance; queries never enter support averaging.

Scope

Closed-set, segmented clips; continuous listening, unknown words and false triggers are not evaluated. Latency covers the encoder only; recognition accuracy after quantization is not reported.

02 Method

Recognize new spoken words from support examples without retraining the encoder.



Core formulation

$$\mathbf{p}_i = \frac{1}{|S_i|} \sum_{(x,y) \in S_i} f_\theta(x)$$

S_i contains support examples of word i; f_θ encodes each clip. Their mean p_i defines that word without updating θ.

$$\hat{y} = \arg\min_i \| f_\theta(x_q) - \mathbf{p}_i \|_2^2$$

Encode query x_q separately and choose the closest word prototype. Query embeddings are never averaged into p_i.

Paper **Code**

03 Results

95%
80-word recognition
English-only base; five supports/word

5.12 ms
small encoder latency
Quantized TinyNet-E; Pixel 7; encoder only

More support handles larger vocabularies

English test vocabulary	Supports per word	Accuracy (%) (Ours)
40 words	1	**90**
80 words	1	**86**
60 words	5	**96**
80 words	5	**95**

English-only base model on held-out MSWC words; 1,000 test episodes. Rounded mean accuracies.

Quantized encoder latency

Device	Small (ms)	Base (ms)
Pixel 7	5.12	78.07
Raspberry Pi 4	6.92	107.13
Jetson Nano	7.32	101.37

Quantized TinyNet-E / EfficientNet-v2 Medium; 200-run mean encoder latency. Excludes log-mel preprocessing and prototype classification.

Takeaway

New support examples add vocabulary without encoder retraining, with accuracy depending on language, vocabulary size and support count. The **5.12 ms** Pixel 7 result measures only the separate small encoder, excluding preprocessing and classification.